REST vs RPC

REST (Representational State Transfer) and RPC (Remote Procedure Call) are two architectural approaches used for designing networked applications, particularly for web services and APIs. Each has its distinct style and is suited for different use cases.

REST (Representational State Transfer)

- **Concept**: REST is an architectural style that uses HTTP requests to access and manipulate data. It treats server data as resources that can be created, read, updated, or deleted (CRUD operations) using standard HTTP methods (GET, POST, PUT, DELETE).
- **Stateless**: Each request from client to server must contain all the necessary information to understand and complete the request. The server does not store any client context between requests.
- **Data and Resources**: Emphasizes on resources, identified by URLs, and their state transferred over HTTP in a textual representation like JSON or XML.
- **Example**: A RESTful web service for a blog might provide a URL like `http://example.com/articles` for accessing articles. A GET request to that URL would retrieve articles, and a POST request would create a new article.

Advantages of REST

- **Scalability**: Stateless interactions improve scalability and visibility.
- **Performance**: Can leverage HTTP caching infrastructure.
- **Simplicity and Flexibility**: Uses standard HTTP methods, making it easy to understand and implement.

Disadvantages of REST

- **Over-fetching or Under-fetching**: Sometimes, it retrieves more or less data than needed.
- **Standardization**: Lacks a strict standard, leading to different interpretations and implementations.

RPC (Remote Procedure Call)

- **Concept**: RPC is a protocol that allows one program to execute a procedure (subroutine) in another address space (commonly on another computer on a shared network). The programmer defines specific procedures.
- **Procedure-Oriented**: Clients and servers communicate with each other through explicit remote procedure calls. The client invokes a remote method, and the server returns the results of the executed procedure.
- **Data Transmission**: Can use various formats like JSON (JSON-RPC) or XML (XML-RPC), or binary formats like Protocol Buffers (gRPC).
- **Example**: A client invoking a method `getArticle(articleId)` on a remote server. The server executes the method and returns the article's details to the client.

Advantages of RPC

- **Tight Coupling**: Allows for a more straightforward mapping of actions (procedures) to server-side operations.
- **Efficiency**: Binary RPC (like gRPC) can be more efficient in data transfer and faster in performance.
- **Clear Contract**: Procedure definitions create a clear contract between the client and server.

Disadvantages of RPC

- **Less Flexible**: Tightly coupled to the methods defined on the server.
- **Stateful Interactions**: Can maintain state, which might reduce scalability.

Conclusion

- **REST** is generally more suited for web services and public APIs where scalability, caching, and a uniform interface are important.
- **RPC** is often chosen for actions that are tightly coupled to server-side operations, especially when efficiency and speed are critical, as in internal microservices communication.

← Previous
Server-Side Caching Vs Client-Side Caching

Next →
Polling Vs Long-Polling Vs Webhooks

☐ Mark as Completed